Homework 3 Report

Instance Segmentation on Colored Medical Images

Student ID: 314561002

GitHub Repository: Visual-Recognition-using-Deep-Learning-HW-3

Visual Recognition using Deep Learning, Spring 2026

1 Introduction

1.1 Task definition

Homework 3 is an instance segmentation task on colored medical images. The dataset contains 209 labeled training/validation images and 101 test images. The target is to segment individual cell instances from four categories, denoted as `class1`, `class2`, `class3`, and `class4`. The official competition metric is AP50, so a predicted instance is counted as correct when the mask IoU with a ground-truth instance is at least 0.50.

The raw dataset is not provided in COCO format. Each training sample is a folder containing `image.tif` and optional class-specific mask files such as `class1.tif`, `class2.tif`, `class3.tif`, and `class4.tif`. A nonzero pixel value in a class mask represents one instance. Therefore, the first part of the solution is a reliable conversion pipeline from raw TIFF masks to COCO-style instance annotations. The test set contains TIFF images and a mapping file from image file names to official image ids.

1.2 Core idea of the method

The final solution is based on a Mask R-CNN-family pipeline implemented with MMDetection [7]. The main idea is not only to train a stronger instance segmentation model, but also to adapt both training and inference to the image structure of this dataset. The images contain many small and crowded cell instances, so full-image resizing can reduce useful local detail and can cause the detector to suppress too many instances. The final system therefore uses:

- a robust TIFF-to-COCO preprocessing pipeline;
- tile-based training so that the detector learns from local crowded views;
- tile-based inference so that small cells are processed at higher effective resolution;
- Cascade Mask R-CNN [3] as the strongest single-model architecture;
- exponential moving average (EMA) training as a stable complementary checkpoint source;
- weighted box fusion (WBF) [4] to combine complementary tiled predictions from multiple models.

The best confirmed public leaderboard result was **0.5658 AP50**, obtained by a three-member WBF ensemble: `exp020` Cascade Mask R-CNN R50-FPN epoch 8, `exp015` Mask R-CNN R50-FPN epoch 21, and `exp023a` Cascade Mask R-CNN R50-FPN with EMA epoch 10. The best local validation AP50 later observed was **0.7816** using horizontal-flip TTA on top of the same ensemble, but this was not recorded as the best confirmed public submission in the project log.

2 Dataset Preprocessing

2.1 Raw data contract

The raw training data uses one folder per image. The input image is stored as `image.tif`; class masks are stored in optional files named `class1.tif` to `class4.tif`. Missing class files are valid and mean that the corresponding class does not appear in that image. Existing mask files can contain sparse instance ids, so the converter cannot assume that instance ids are dense or consecutive.

The test data is stored as flat TIFF files under `data/test_release`. The file `data/test_image_name_to_ids` maps each test file name to the official image id used in the submission JSON.

2.2 Mask-to-instance conversion

For each image and class mask, the converter performs the following steps:

1. Load the raw image and class mask.
2. Find all unique nonzero pixel values in the class mask.
3. For each unique value, create a binary instance mask.
4. Compute the instance bounding box in `xywh` format.
5. Encode the binary mask using COCO-compatible RLE.
6. Add an annotation record with stable category id mapping from `class1..class4` to category ids `1..4`.

A key design choice is that instance extraction is based on the actual unique nonzero values in each mask, not on a loop over assumed ids such as `1..N`. This avoids losing instances when the mask uses sparse ids.

2.3 Data integrity checks

Before model training, the pipeline checks the following conditions:

- image and mask dimensions match;
- missing class masks are handled as zero-instance classes;
- empty masks do not create invalid annotations;
- category ids remain in the valid range `1..4`;
- every bounding box has positive width and height;
- every encoded mask has nonzero area;
- every validation/test image id is mapped correctly;
- submission masks are decodable compressed RLE.

These checks are important because a silent conversion bug would directly reduce AP50, especially on small or rare instances.

2.4 Tile dataset generation

The strongest training line uses a derived tiled COCO training dataset. Each full image is split into `768 x 768` tiles. The best tile-training configuration uses zero tile overlap for training, drops empty tiles, and keeps an instance only if at least 50% of its visible area remains inside the tile. More precisely, the main tile-training settings are:

- tile size: `768 x 768`;
- train tile overlap: `0.0`;
- drop empty tiles: true;
- minimum visible fraction: `0.5`;
- minimum ground-truth area: `1.0`;
- class ids: unchanged from the original four-class mapping.

The hypothesis is that tile-aligned training exposes the detector to the same local scale and density that it will see during tiled inference. It may improve crowded-instance recall because the model learns from local views where cells occupy a larger number of pixels. It may fail if tiling removes too much context or over-represents local fragments. In this project, tile-aligned training was one of the most important positive changes.

3 Validation and Evaluation Protocol

3.1 Metric

The official metric is AP50. In local evaluation, predictions are exported into COCO-compatible format and evaluated with COCOEval [8]. The main validation metric reported in this report is `coco/segm_mAP_50`. AP and AP75 are also tracked when useful, because a change can improve AP50 while hurting tighter mask quality.

3.2 Initial split and later validation hardening

Early experiments used a fixed fold-0 validation split to build a trusted baseline quickly. This was necessary because the dataset has only 209 labeled images, and repeated cross-validation was too expensive for every experiment. However, the project also created stronger validation assets later, including stratified 5-fold splits and a 42-image test-like diagnostic holdout. These were designed to account for image-level class presence, instance counts, image geometry, object density, tiny images, and extreme aspect ratios.

The validation policy was progressive:

- early stage: use one fixed split to verify training, evaluation, and export;
- serious comparison stage: compare class presence, instance count, image size, and density distributions;
- late stage: use public leaderboard only for bounded final checks, not broad uncontrolled sweeps.

This policy was influenced by previous homework mistakes. In HW1, the public score plateaued partly because late-stage decisions relied too much on a tiny noisy validation set and highly correlated models. In HW2, the strongest model family was found too late, and export parity checks were also done late. For HW3, the strategy was to build a trusted pipeline early, keep the experiment log detailed, and avoid broad low-ROI changes.

4 Model Architecture

4.1 Baseline: Mask R-CNN with FPN

The first baseline is Mask R-CNN [1] with a ResNet-50 backbone and Feature Pyramid Network (FPN) [2]. Mask R-CNN is a two-stage instance segmentation model. Its main components are:

- **Backbone:** ResNet-50 extracts feature maps from the input image.
- **Neck:** FPN combines multi-scale features so both small and larger objects can be detected.
- **RPN:** Region Proposal Network proposes candidate object regions.
- **RoI head:** The bounding-box head classifies proposals and refines boxes.
- **Mask head:** A small fully convolutional head predicts a binary mask for each positive RoI.

This family matches the homework guidance and provides a stable base for modifying components.

4.2 Final single-model architecture: Cascade Mask R-CNN

The strongest single model is Cascade Mask R-CNN R50-FPN. Cascade Mask R-CNN extends the standard two-stage detector with multiple bounding-box refinement stages. In this project, the cascade uses three bbox heads with increasing IoU thresholds:

- stage 1 IoU threshold: 0.5;
- stage 2 IoU threshold: 0.6;
- stage 3 IoU threshold: 0.7;
- stage loss weights: 1.0, 0.5, 0.25;
- number of foreground classes: 4;
- mask head classes: 4.

The hypothesis is that cascade refinement improves proposal quality and score ranking, which is useful for crowded cell images where many instances are close together. It may fail if the extra stages overfit the small dataset or increase inference noise. In the experiment log, the Cascade model improved the public leaderboard from the previous tile-trained Mask R-CNN line and became the best confirmed single model.

4.3 EMA branch

The `exp023a` branch keeps the same Cascade Mask R-CNN R50-FPN architecture but adds an exponential moving average (EMA) of model weights using MMEngine's EMA hook. EMA maintains a smoothed copy of the training weights. The hypothesis is that EMA improves stability and generalization by reducing checkpoint noise. It may fail if smoothing removes useful late training adaptation or if it produces a model too similar to the non-EMA checkpoint.

In this project, the EMA checkpoint was useful as an ensemble member. It did not replace the final system by itself, but it added complementary signal to the existing `exp020 + exp015` ensemble.

4.4 Other architecture branches

Several additional branches were tested or prepared:

- **ConvNeXt-T Mask R-CNN:** tested as a stronger backbone branch, but it did not replace the R50 anchor locally.
- **ResNet-101 COCO-pretrained Mask R-CNN:** strong local result but weak public leaderboard transfer.
- **Hybrid Task Cascade (HTC):** prepared as a possible diverse member, but later treated as legacy because it underperformed locally.
- **RTMDet-Ins:** implemented as a high-risk one-stage instance segmentation branch, but memory and stability issues prevented it from becoming part of the final confirmed solution.
- **Cascade ConvNeXt-T:** prepared after the best ensemble result, but no final run was recorded in the current report state.

These branches were useful because they tested whether model-family or backbone changes mattered more than the tile pipeline. The final evidence suggests that the tile pipeline and complementary ensembling mattered more than simply using a larger backbone.

5 Training Setup

5.1 Environment

The training environment uses MMDetection 3.3.0, MMEngine 0.10.7, MMCV 2.1.0, PyTorch 2.1.0 with CUDA 11.8, TorchVision 0.16.0, Python 3.10, and MMPretrain 1.2.0. A dedicated conda environment named `hw3-mmdet` is used to avoid dependency conflicts.

5.2 Common hyperparameters

Table 1 summarizes the main settings used by the final model family.

Table 1: Common training and inference settings for the final model family.

Item	Setting
Framework	MMDetection 3.3.0
Architecture	Cascade Mask R-CNN R50-FPN for the best single model
Classes	4 foreground classes
Train data	Fold-0 tile-only train dataset for main diagnostic runs
Train tile size	`768 x 768`, overlap `0.0`
Train augmentation	Random horizontal flip only for the main tile recipe
Schedule	24 epochs for the main Mask R-CNN/Cascade lines
Batch handling	Physical batch size 1 for memory-heavy Cascade runs, with gradient accumulation 2
Validation during training	Disabled in some Cascade runs for laptop stability; checkpoints evaluated afterward
Inference tile size	`768 x 768`
Inference overlap	`0.25`
Score threshold	`0.03` for the strongest single model and ensemble members
Max predictions per image	`1000`
NMS threshold	`0.5` before final WBF merge
Final ensemble merge	Class-aware WBF, merge IoU `0.60`

5.3 Handling memory constraints

The main hardware constraint was limited GPU memory. I handled this by using batch size 1 for heavier models, gradient accumulation, fewer workers for stability, and tile-based training. Validation inside the training loop was disabled for some Cascade runs because repeated in-loop validation caused driver-level instability. Instead, checkpoints were saved and evaluated afterward with the project inference scripts.

6 Inference and Submission Generation

6.1 Full-image inference bottleneck

The first major inference bottleneck was the maximum number of predictions per image. On crowded cell images, the default `max_per_img = 100` removed many valid detections. Raising this cap to 300 increased local AP50 substantially on the early R50 baseline and improved the public leaderboard. This showed that the dataset is dense enough that default detection caps are too restrictive.

6.2 Sliding-window tiled inference

The strongest inference change was tiled inference. The model is applied to overlapping `768 x 768` windows with 25% overlap. Predictions from tiles are mapped back to full-image coordinates, merged by class-aware NMS or WBF, and encoded as full-image RLE masks.

The hypothesis is that tiled inference improves small-object recall by reducing the effective downscaling of local cell structures. It may fail if tile borders create duplicate objects, fragment masks, or remove context. In practice, tiled inference gave the largest early public leaderboard jump.

6.3 Submission JSON

Each prediction record contains:

- `image_id`;
- `category_id`;
- `bbox` in `xywh` format;
- `score`;
- `segmentation` as compressed RLE.

The submission guard validates that JSON is valid, category ids are in range, image ids exist in the test mapping, masks can be decoded, and decoded masks have nonzero area.

7 Ensembling

7.1 Weighted box fusion strategy

The best final model is an ensemble. Each member first produces tiled full-image predictions. The ensemble then performs class-aware WBF at merge IoU 0.60. Box coordinates are fused by score-weighted averaging. The segmentation mask for each fused cluster is taken from the highest-score matched prediction. This is simpler than mask-level fusion but avoids introducing unstable mask averaging artifacts.

The final confirmed public ensemble contains:

- `exp020`: Cascade Mask R-CNN R50-FPN, epoch 8, weight 1.0;
- `exp015`: Mask R-CNN R50-FPN tiled training, epoch 21, weight 1.0;
- `exp023a`: Cascade Mask R-CNN R50-FPN with EMA, epoch 10, weight 0.75.

The hypothesis is that these models are complementary because they share the strong tiled pipeline but differ in architecture and checkpoint smoothing. WBF can recover objects detected by only one member while reducing duplicate boxes. It may fail if models are too correlated or if WBF merges nearby true instances incorrectly. The local and public results show that the chosen members were complementary enough to improve performance.

8 Experiments and Results

8.1 Result summary

Table 2 summarizes the most important score milestones. The final public leaderboard score is from the best confirmed submission in the project log.

Table 2: Main public leaderboard milestones.

Experiment	Public AP50	Main idea
`exp011`	0.4526	Class-balanced R50 Mask R-CNN, full-image inference
`exp013/exp011 tile`	0.5130	Tiled inference on R50 line
`exp015`	0.5286	Tile-only training plus tiled inference
`exp020`	0.5378	Cascade Mask R-CNN R50-FPN, epoch 8
`ens001`	0.5537	WBF ensemble of `exp020` and `exp015`
`ens003`	**0.5658**	WBF ensemble of `exp020`, `exp015`, and `exp023a` EMA

8.2 Early Mask R-CNN and inference calibration

The initial Mask R-CNN R50-FPN baseline established a working train/evaluate/export pipeline. Early public scores were around the strong-baseline range, but the model was limited by crowded-instance

truncation. Increasing `max_per_img` from 100 to 300 improved fold-0 AP50 from 0.462 to 0.630 at score threshold 0.05, and to 0.634 at score threshold 0.03. Both calibrated single-scale submissions reached around 0.45 public AP50. This result showed that recall loss from prediction caps was a real bottleneck.

8.3 Class balancing

The class distribution was long-tailed, especially for `class3` and `class4`. A train-only class-balanced sampling branch improved local AP50 from the calibrated R50 anchor to 0.660 and reached 0.4526 on the public leaderboard. The result was positive, but later experiments showed that tiling was a much larger lever than class balancing alone.

8.4 Tiled inference

Tiled inference was the first decisive breakthrough. Table 3 summarizes the validation sweep on the class-balanced R50 checkpoint.

Table 3: Tiled inference validation sweep on the R50 class-balanced checkpoint.

Inference setting	Local AP50	AP	AP75
Full image, max 300	0.660	0.323	0.517
Tile 768, overlap 0.25, max 500	0.729	0.332	0.564
Tile 768, overlap 0.25, max 1000	**0.738**	0.331	**0.569**
Tile 512, overlap 0.25, max 1000	0.737	0.319	0.571
Tile 1024, overlap 0.25, max 1000	0.705	0.328	0.549

The best overall validation setting was `768 x 768` tiles with 25% overlap and a final cap of 1000 predictions per image. It also transferred to the public leaderboard, improving public AP50 to about 0.5130. This confirmed that the main issue was not only the model architecture, but also the effective inference resolution and dense-instance handling.

8.5 Tile-aligned training

After tiled inference worked, the next hypothesis was that training should match the tiled inference regime. The `exp015` branch trained on a materialized tile-only COCO dataset using `768 x 768` tiles. It reached local AP50 0.7201 with the winning tiled inference setting and improved the public leaderboard to 0.5286.

This was an important result because it showed that tile alignment was useful both at inference time and during training. A later all-209 mixed full+tile refit did not improve public performance, suggesting that simply using all labels or mixing full-image samples was not automatically better than the clean tile-only recipe.

8.6 Cascade Mask R-CNN

The `exp020` Cascade Mask R-CNN branch used the same tile-only training and tiled inference pipeline as `exp015`, but changed the architecture from Mask R-CNN to Cascade Mask R-CNN. A checkpoint sweep found that the best checkpoint was early, at epoch 8. Table 4 shows the key sweep results.

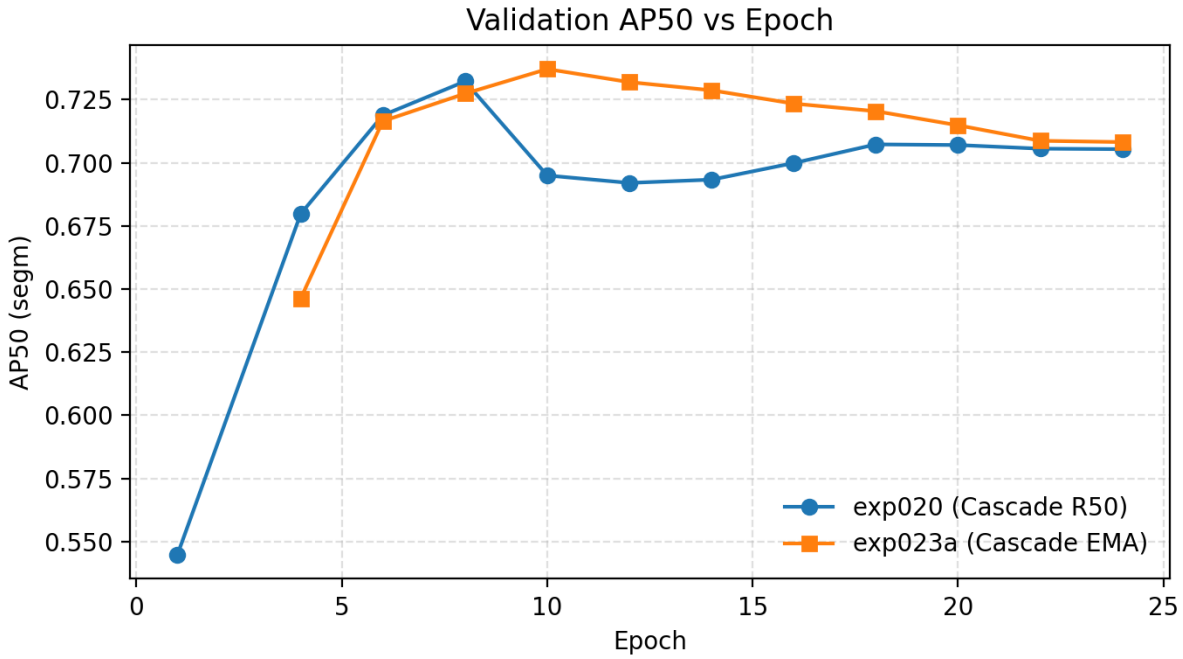


Table 4: Cascade Mask R-CNN epoch sweep with tiled validation.

Checkpoint	Local AP50
Epoch 1	0.5448
Epoch 6	0.7188
Epoch 8	**0.7322**
Epoch 18	0.7073
Epoch 20	0.7070
Epoch 24	0.7054

Epoch 8 improved over the `exp015` local anchor by 0.0121 AP50 and reached 0.5378 on the public leaderboard. This became the strongest confirmed single-model result.

A score-threshold sweep showed that the original score threshold 0.03 remained best for this checkpoint. Raising the threshold to 0.05 and 0.07 reduced AP50 to 0.7290 and 0.7252, respectively. This means the model still benefited from retaining lower-confidence predictions.

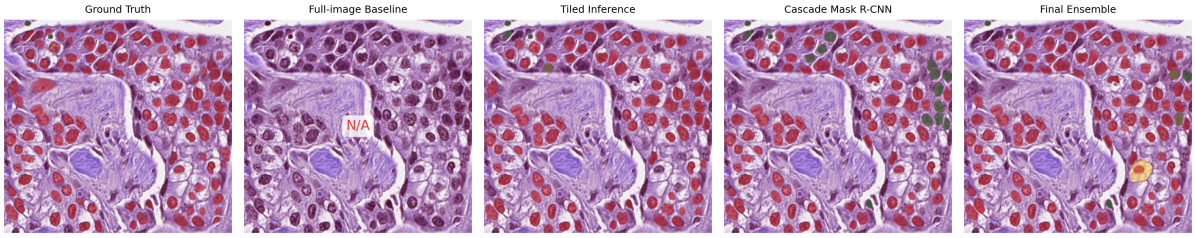
8.7 Two-model ensemble

The next experiment combined the two strongest tiled models: `exp020` Cascade Mask R-CNN epoch 8 and `exp015` Mask R-CNN epoch 21. Equal-weight WBF with merge IoU 0.60 achieved 0.7612 local AP50 and 0.5537 public AP50. This improved local validation by 0.0290 over the Cascade single model and improved public AP50 by 0.0159.

The result indicates that the two models were not fully correlated. Even though both used ResNet-50-FPN and the same tile inference policy, the difference between standard Mask R-CNN and Cascade Mask R-CNN created useful complementary detections.

8.8 EMA and three-model ensemble

The EMA branch `exp023a` produced a strong smoothed Cascade checkpoint. The best recorded checkpoint, epoch 10, reached 0.7371 local AP50. It was then added as a third ensemble member to the



existing `exp020 + exp015` pair.

The final submitted ensemble `ens003` used weights 1.0, 1.0, and 0.75 for `exp020`, `exp015`, and `exp023a`, respectively. It achieved 0.5658 public AP50, the best confirmed public score in the project log. A nearby weight sweep showed that reducing the EMA weight to 0.60 gave only a tiny local AP50 increase but lower AP/AP75, so the submitted 0.75 weight was kept.

8.9 Pseudo-labeling

Pseudo-labeling was tested conservatively using the strong `ens003` teacher. The filter kept 11059 of 26843 teacher predictions using score, area, per-image cap, and NMS constraints. The filtered pseudo-label set was class-skewed: 5258 class1, 5448 class2, 169 class3, and 184 class4 predictions.

The pseudo-labeled student did not beat the best single-model anchors. Its best checkpoint reached 0.7106 local AP50, below `exp020` epoch 8 and `exp023a` epoch 10. As a low-weight ensemble member, it gave only a very small improvement: `ens004_b` improved AP50 by 0.0005 over `ens003`. This is too small to treat as a major contribution. The implication is that pseudo-labeling may need better support-aware filtering or class-balanced filtering before it becomes useful.

8.10 TTA

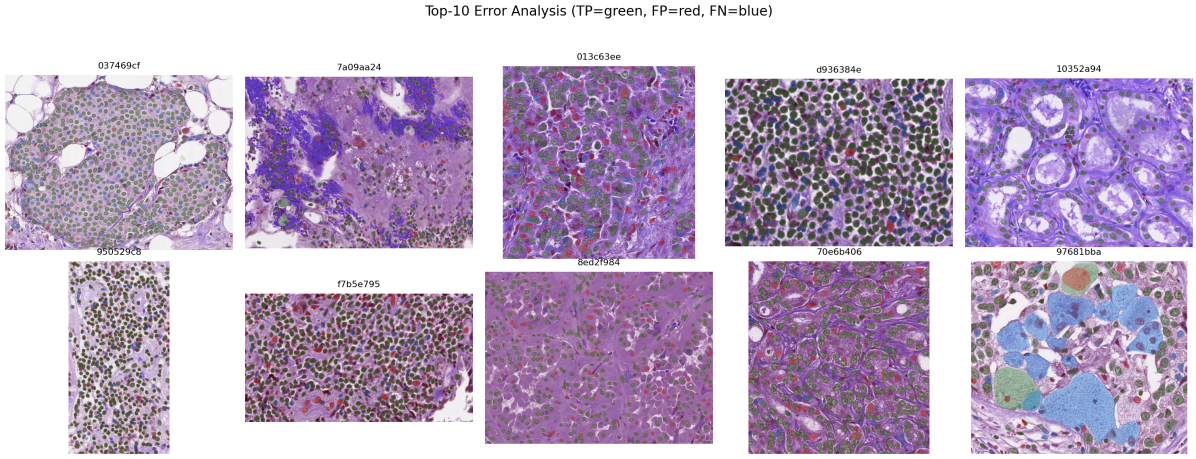
Horizontal/vertical flip TTA was tested on the best `ens003` ensemble. Full hflip+vflip TTA improved AP50 but reduced AP and AP75, so it was not recommended. Hflip-only TTA was stronger: it reached 0.7816 local AP50, 0.3582 AP, and 0.6196 AP75. This was the best local ensemble result in the current log, but the best confirmed public submission remains the non-TTA `ens003` score of 0.5658.

8.11 Negative or low-value experiments

Several experiments did not become part of the final confirmed system:

- **ConvNeXt-T Mask R-CNN:** viable but did not beat the calibrated R50 anchor locally.
- **R101 COCO-pretrained Mask R-CNN:** locally strong but public score regressed, showing local split mismatch.
- **All-209 mixed full+tile refit:** public score decreased relative to `exp015`; using more data was not automatically better when the recipe changed.
- **Padded tile context:** `1024` context with `768` valid center dropped local AP50 to 0.5036, likely because the ownership rule was too destructive.
- **Morphology:** dilation and erosion sharply reduced AP/AP75; one-pixel closing was essentially tied and not worth a public submission.
- **RTMDet-Ins:** implementation was prepared, but memory and driver failures prevented it from becoming a confirmed result.

These negative results were useful because they prevented the final strategy from over-investing in unstable or low-transfer changes.



9 Discussion

9.1 What worked

The most effective improvements were directly connected to the structure of the dataset. The images are crowded and contain many small cell instances, so tiled inference gave a large recall and AP50 improvement. Tile-aligned training then improved the model further by matching the train-time and test-time views. Cascade Mask R-CNN improved the strongest single-model line by refining bounding boxes and scores. Finally, WBF ensembling improved performance because the selected models had complementary predictions.

9.2 What did not work as expected

Some ideas that are usually strong did not transfer well here. Larger or different backbones did not automatically improve public AP50. All-data refitting was also risky because it removed clean validation and changed the effective training recipe. Pseudo-labeling was weaker than expected because the pseudo-label distribution was skewed and lacked support counts from the ensemble. Simple morphology changed mask areas too much and harmed tighter mask quality.

9.3 Implications for private leaderboard performance

The public/private distributions were described as similar in the homework slides, so public leaderboard movement was treated as meaningful but not absolute. The final system is not based on a single public-only trick; it combines changes that also improved local validation: tile inference, tile training, Cascade architecture, EMA, and WBF. This makes the final public score more likely to generalize than a narrow threshold-only leaderboard optimization.

10 Conclusion

This project built an instance segmentation system for colored medical cell images using a Mask R-CNN-family pipeline. The final confirmed submission used a three-member tiled WBF ensemble consisting of Cascade Mask R-CNN, Mask R-CNN, and an EMA-smoothed Cascade checkpoint. The most important finding was that dense-cell segmentation performance depended strongly on tile-aware training and inference. The best confirmed public leaderboard result was **0.5658 AP50**, while later local hflip-only TTA reached **0.7816 local AP50** and can be considered as a possible final-stage update if it is submitted and verified.

The main lesson is that, for this dataset, model architecture alone was not enough. The largest gains came from matching the model pipeline to the data: preserving small-instance detail, avoiding crowded-instance truncation, and combining complementary predictions only after strong single-model lines were established.

References

[1] K. He, G. Gkioxari, P. Dollar, and R. Girshick. Mask R-CNN. In *Proceedings of the IEEE International Conference on Computer Vision (ICCV)*, 2017.

[2] T.-Y. Lin, P. Dollar, R. Girshick, K. He, B. Hariharan, and S. Belongie. Feature Pyramid Networks for Object Detection. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2017.

[3] Z. Cai and N. Vasconcelos. Cascade R-CNN: Delving into High Quality Object Detection. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2018.

[4] R. Solovyev, W. Wang, and T. Gabruseva. Weighted Boxes Fusion: Ensembling Boxes from Different Object Detection Models. *Image and Vision Computing*, 107:104117, 2021.

[5] Z. Liu, H. Mao, C.-Y. Wu, C. Feichtenhofer, T. Darrell, and S. Xie. A ConvNet for the 2020s. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2022.

[6] C. Lyu, W. Zhang, H. Huang, Y. Zhou, Y. Wang, Y. Liu, S. Zhang, and K. Chen. RTMDet: An Empirical Study of Designing Real-Time Object Detectors. *arXiv preprint arXiv:2212.07784*, 2022.

[7] K. Chen, J. Wang, J. Pang, Y. Cao, Y. Xiong, X. Li, S. Sun, W. Feng, Z. Liu, J. Xu, Z. Zhang, D. Cheng, C. Zhu, T. Cheng, Q. Zhao, B. Li, X. Lu, R. Zhu, Y. Wu, J. Dai, J. Wang, J. Shi, W. Ouyang, C. C. Loy, and D. Lin. MMDetection: Open MMLab Detection Toolbox and Benchmark. *arXiv preprint arXiv:1906.07155*, 2019. https://github.com/open-mmlab/mmdetection

[8] T.-Y. Lin, M. Maire, S. Belongie, J. Hays, P. Perona, D. Ramanan, P. Dollar, and C. L. Zitnick. Microsoft COCO: Common Objects in Context. In *European Conference on Computer Vision (ECCV)*, 2014. https://cocodataset.org/